

Interface Kē Hua Interface.
interface L { void drawLine(); }
interface P extends L { void drawPolygon(); }

Q1:

```
interface Mail {  
    void sendMail();  
    void log();  
}  
class MailImpl implements Mail {  
    @Override  
    void sendMail() {  
        System.out.println("gu Mail");  
    }  
    @Override  
    void log() {  
        System.out.println("gu log");  
    }  
}  
class Main {  
    public static void main(String[] args) {  
        MailImpl sc1 = new MailImpl();  
        sc1.sendMail();  
        sc2 send sc1.log();  
    }  
}
```

```

B2 public interface Shape {
    String color = "Red";
    void draw();
}

Shape {
    @Override
    void draw() {
        System.out.println("Draw" +
        color + "Rectangle");
    }
}

```

class circle implements shape {

@ Override .

public void draw () {

System.out.println("Draw " + color +
circle"); } }

class ~~sha~~ main {

public static void main (String[] args) {

Shape rect = new Rectangle();

rect.draw ();

System.out.println ("..");

Shape circle = new Circle();

circle.draw ();

} }

- Bg: abstract class PTDC {
 protected String baiPT;
 protected String hangSX;
 String layTHSX() {
 return hangSX;
 }
 void batDau ();
 void tangLoc ();
 void dungai ();
 abstract double layVanToc ();
 }

class HangSX {
 private String tenHangSX;
 private String tenAG;
 }

class MayBay extends PTDC {
 private String baiNhanLieu;
 void catCanh () {
 System.out.println("May bay");
 }

void haCanh () {
 System.out.println("May bay");
 }

@ Override


```
double layVantoc () {  
    return layVantoc;  
}
```

```
Class OTO extends PTDC {  
    private String loanNK;
```

```
    @Override
```

```
    double layVantoc () {  
        return layVantoc;  
    }
```

```
}
```

```
Class XeDap extends PTDC {
```

```
    @Override
```

```
    double layVantoc () {
```

```
        return layVantoc;  
    }
```

```
class Main {
```

```
    public static void main (String [] args)
```

```
    {  
        PTDC s1 = new Maybay;
```

```
        PTDC s2 = new OTO;
```

```
        PTDC s3 = new XeDap;
```

```
        May s1 . batDan ();
```

```
        s2 . batDan ();
```

```
        s1 . DangLai ();  
    }
```

```
}
```

Class

```

14 abstract class Car {
    protected String color;
    protected double speed;
    public void Car (String color, double
speed) {

```

```

        this.color = color;

```

```

        this.speed = speed;
    }

```

```

    public String get color () {
        return color;
    }

```

```

    String getSpeed () {
        return speed;
    }

```

```

    abstract void accelerate ();
}

```

```

class BMW extends Car {

```

```

    public BMW (String color, double speed) {
        super (color, speed);
    }

```

```

    @Override

```

```

    public void accelerate () {

```

```

        System.out.println("xun dang tai gao");

```

```

        speed += 20;

```

```

        System.out.println("vi hien tai: " + speed + "km/h");
    }

```


@Override

String toString() {

return "mau" + color + "vtec" + speed;

}
class Civic extends car {

public Civic (String color, double speed) {

super (color, speed);

}
@Override

void accelerate () {

speed += 25;

}
@Override

String toString() {

return "mau" + color + "vtec" + speed;

}
}

class Main {

public static void main (String[] args) {

BmwCar c1 = new Bmw ("Red", 80);

CivicCar c2 = new Civic ("white", 80);

c1.accelerate();

c2.accelerate();

}
}

BTS

```
interface Moveable {  
    void move up ();  
    void movedown ();  
    void move Left ();  
    void move Right ();  
}
```

```
class MoveableCircle implements Moveable {
```

```
    int radius;
```

```
    Moveable Point center;
```

```
    @Override
```

```
    void move up () {
```

```
        Syst center . move up ();
```

```
    }  
    void movedown () {
```

```
        center . movedown ();
```

```
    }
```

```
    void move Left () {
```

```
        center . move Left ();
```

```
    }
```

```
    void move Right () {
```

```
        center . move Right ();
```

```
    }  
    public String toString () {
```

```
        return "circle center " + center + "
```

ClassMate


```
radius + "Radius";  
    }  
}
```

```
class MovablePoint implements Movable {  
    int x, y, xSpeed, ySpeed;  
    void Movable(int x, int y, int xSpeed,  
int ySpeed) {
```

```
        this.x = x;
```

```
        this.y = y;
```

```
        this.xSpeed = xSpeed;
```

```
        this.ySpeed = ySpeed;
```

```
    }
```

```
    public void moveUp() {
```

```
        y -= ySpeed;
```

```
    }
```

```
    public void moveDown() {
```

```
        y += ySpeed;
```

```
    }
```

```
    public void moveLeft() {
```

```
        x -= xSpeed;
```

```
    }
```

```
    String toString() {
```

```
        return "Point";
```

```
    }  
}
```